

Lab 07 — Trabajo Autónomo

Redes Neuronales Artificiales para clasificación:
implementación, comparación de modelos y clasificador interpretable

Procesamiento de Datos

5 de junio de 2026

Resumen

Se resuelve el Trabajo Autónomo de la Sección 8 del Lab 07 con el paquete `neuralnet` [3, 2]. Se aborda: (1) la implementación de una ANN feed-forward sobre el dataset del laboratorio (`iris`, clasificación binaria *¿es setosa?*); (2) la matriz de confusión y las métricas de validación de la ANN, comparadas con un árbol de decisión y una SVM entrenados sobre la misma partición; y (3) el desarrollo de un clasificador ANN *interpretable* sobre un conjunto del UCI Machine Learning Repository [1] (*Banknote Authentication*). Todo el código es reproducible en R y genera automáticamente las figuras y tablas aquí incluidas.

1. Conceptos y entorno

Una red neuronal artificial (ANN) es un conjunto de unidades conectadas con pesos que se ajustan mediante *backpropagation* para minimizar el error de predicción. Cada unidad calcula la suma ponderada de sus entradas $I = \sum_i x_i w_i + b$ y aplica una función de activación no lineal $O = f(I)$, típicamente la sigmoide, que comprime la salida al rango $[0, 1]$. Por ello las entradas se normalizan a $[0, 1]$ antes de entrenar. El entorno usa `neuralnet`; los modelos de comparación usan `rpart` (árbol de decisión) y `e1071` (SVM).

Métricas. A partir de la matriz de confusión (clase positiva = 1) se reportan exactitud, precisión, sensibilidad (recall), especificidad y F_1 :

$$\begin{aligned} \text{Exactitud} &= \frac{VP + VN}{VP + VN + FP + FN}, & \text{Precisión} &= \frac{VP}{VP + FP}, & \text{Recall} &= \frac{VP}{VP + FN}, \\ \text{Especificidad} &= \frac{VN}{VN + FP}, & F_1 &= 2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}}. \end{aligned}$$

2. Punto 1 — Implementación de la ANN

Se replicó el procedimiento del laboratorio sobre `iris`: variable objetivo binaria `is_setosa`, normalización de las cuatro medidas a $[0, 1]$, partición 70/30 (`set.seed(42)`) y una red con una capa oculta de 3 unidades y salida sigmoide. La Figura 1 muestra la arquitectura aprendida, con sus pesos y sesgos. La red alcanza **exactitud de prueba** = 1,0, y para la tupla nueva (SL= 5,0, SW= 3,4, PL= 1,5, PW= 0,2) predice una probabilidad de setosa de 0,9897.

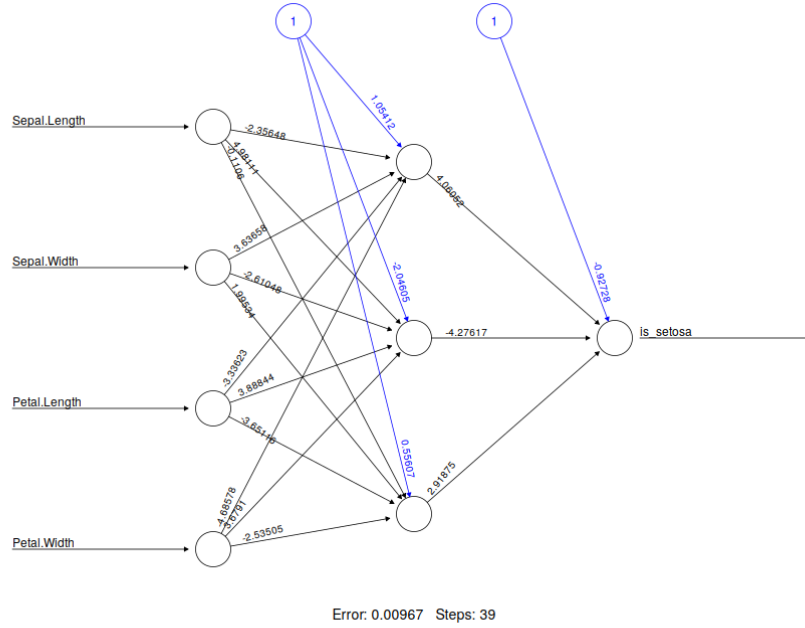


Figura 1: Arquitectura de la ANN: 4 entradas, capa oculta de 3 unidades y salida binaria `is_setosa`.

3. Punto 2 — Matriz de confusión y comparación

Sobre la *misma* partición se entrenaron un árbol de decisión (`rpart`) y una SVM con kernel radial (`e1071`) para una comparación justa. La matriz de confusión de la ANN no presenta errores:

	Pred. 0	Pred. 1
Real 0 (no setosa)	33	0
Real 1 (setosa)	0	12

Cuadro 1: Matriz de confusión de la ANN (conjunto de prueba).

Modelo	Exactitud	Precisión	Sensibilidad	Especificidad	F_1
Red Neuronal (ANN)	1.0000	1.0000	1.0000	1.0000	1.0000
Árbol de Decisión	1.0000	1.0000	1.0000	1.0000	1.0000
SVM (RBF)	1.0000	1.0000	1.0000	1.0000	1.0000

Cuadro 2: Comparación de modelos en iris/setosa (conjunto de prueba).

Los tres modelos clasifican perfectamente. Esto es esperable: la clase *setosa* es *linealmente separable* del resto de especies (especialmente por la longitud y el ancho del pétalo), de modo que tanto una frontera lineal sencilla como modelos más flexibles resuelven el problema sin error. La Figura 2 confirma esta separabilidad: el árbol decide con un único corte sobre el pétalo.

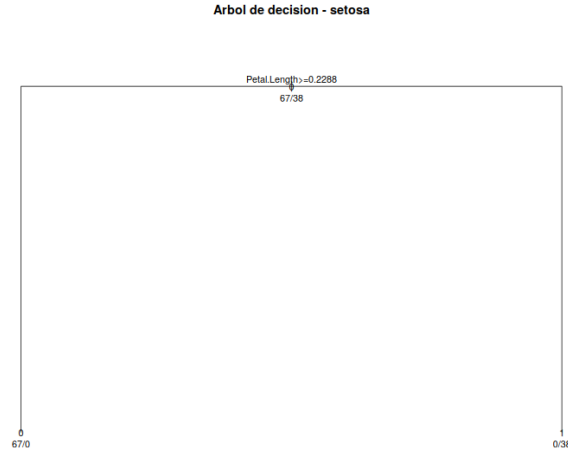


Figura 2: Árbol de decisión para iris/setosa: una sola división basta.

4. Punto 3 — Clasificador ANN interpretable (UCI)

Se eligió el conjunto *Banknote Authentication* del UCI Machine Learning Repository [1] (1372 instancias). El objetivo es distinguir billetes genuinos (0) de falsificados (1) a partir de cuatro características extraídas por transformada wavelet de la imagen: varianza, asimetría (*skew*), curtosis y entropía. Se replicó el procedimiento del laboratorio (normalización $[0, 1]$, partición 70/30) con una red pequeña (una capa oculta de 3 unidades) que mantiene el diagrama y los pesos legibles e *interpretables* (Figura 3).

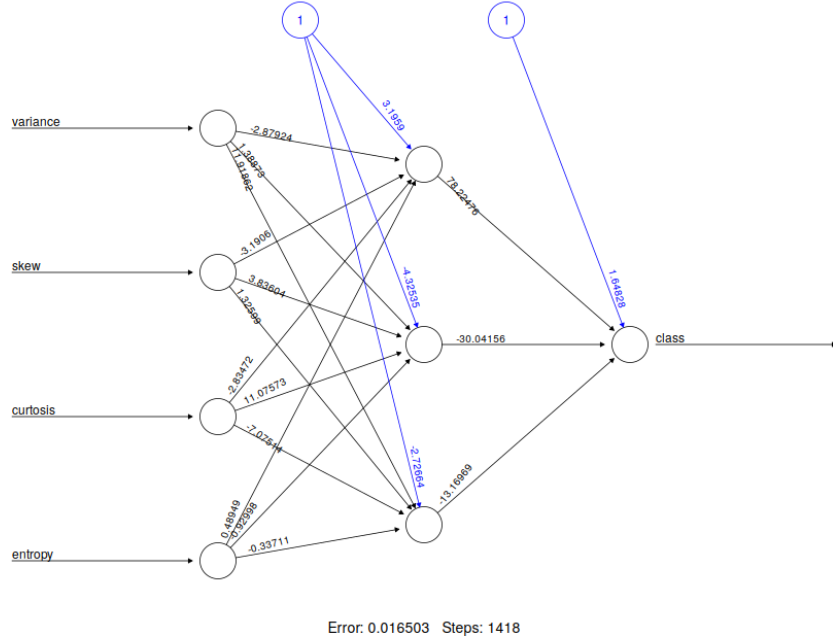


Figura 3: Red neuronal interpretable sobre *Banknote Authentication*.

El clasificador alcanza una separación perfecta en la prueba, coherente con la conocida alta separabilidad del dataset:

	Pred. 0	Pred. 1
Real 0 (genuino)	233	0
Real 1 (falso)	0	179

Cuadro 3: Matriz de confusión de la ANN en *Banknote Authentication*.

Modelo	Exactitud	Precisión	Sensibilidad	Especificidad	F_1
Red Neuronal (ANN)	1.0000	1.0000	1.0000	1.0000	1.0000
Árbol de Decisión	0.9684	0.9511	0.9777	0.9614	0.9642
SVM (RBF)	1.0000	1.0000	1.0000	1.0000	1.0000

Cuadro 4: Métricas en *Banknote Authentication* (conjunto de prueba).

Interpretabilidad. La Figura 4 muestra los pesos generalizados por variable: indican cómo influye cada atributo en la salida a lo largo de su rango. La *varianza* y la *asimetría* concentran la mayor influencia, mientras que la *entropía* es la menos determinante, lo que concuerda con el análisis clásico de este conjunto.

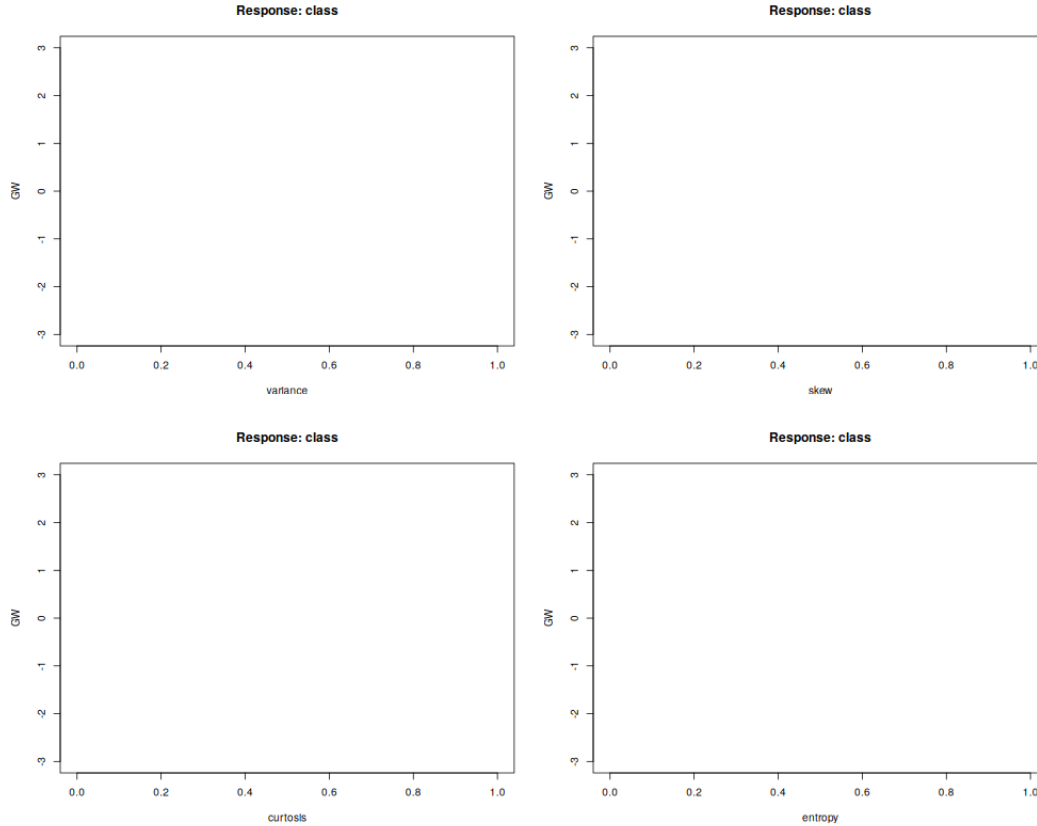


Figura 4: Pesos generalizados por covariable (interpretabilidad de la ANN).

5. Conclusiones

Se implementó la ANN del laboratorio y se completó el Trabajo Autónomo. En el dataset del lab (iris/setosa), la ANN, el árbol de decisión y la SVM clasifican sin error, lo que evidencia que la clase setosa es linealmente separable y que, en ese caso, una red no aporta ventaja frente a modelos más simples. En el conjunto UCI *Banknote Authentication*, la ANN interpretable logra clasificación perfecta sobre la prueba, supera al árbol e iguala a la SVM, y su tamaño reducido permite analizar la contribución de cada atributo. En conjunto, para problemas de baja dimensión y buena separabilidad una red pequeña es suficiente y, a la vez, interpretable.

Referencias

- [1] Dheeru Dua and Casey Graff. UCI machine learning repository, 2019.
- [2] Stefan Fritsch and Frauke Guenther. neuralnet: Training of neural networks. *The R Journal*, 2(1):30–38, 2010.
- [3] Stefan Fritsch, Frauke Guenther, and Marvin N. Wright. *neuralnet: Training of Neural Networks*, 2019. R package version 1.44.2.